

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

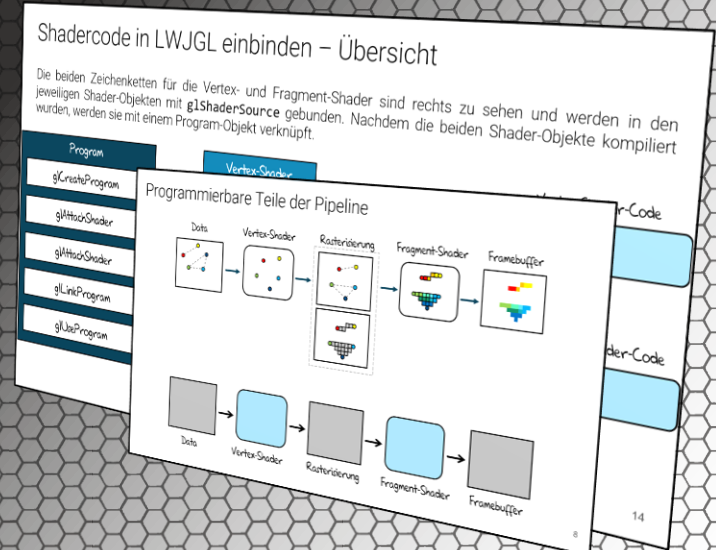
Sommersemester 2026



Realisierung des Blur-Effekts mit einem Shader

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz



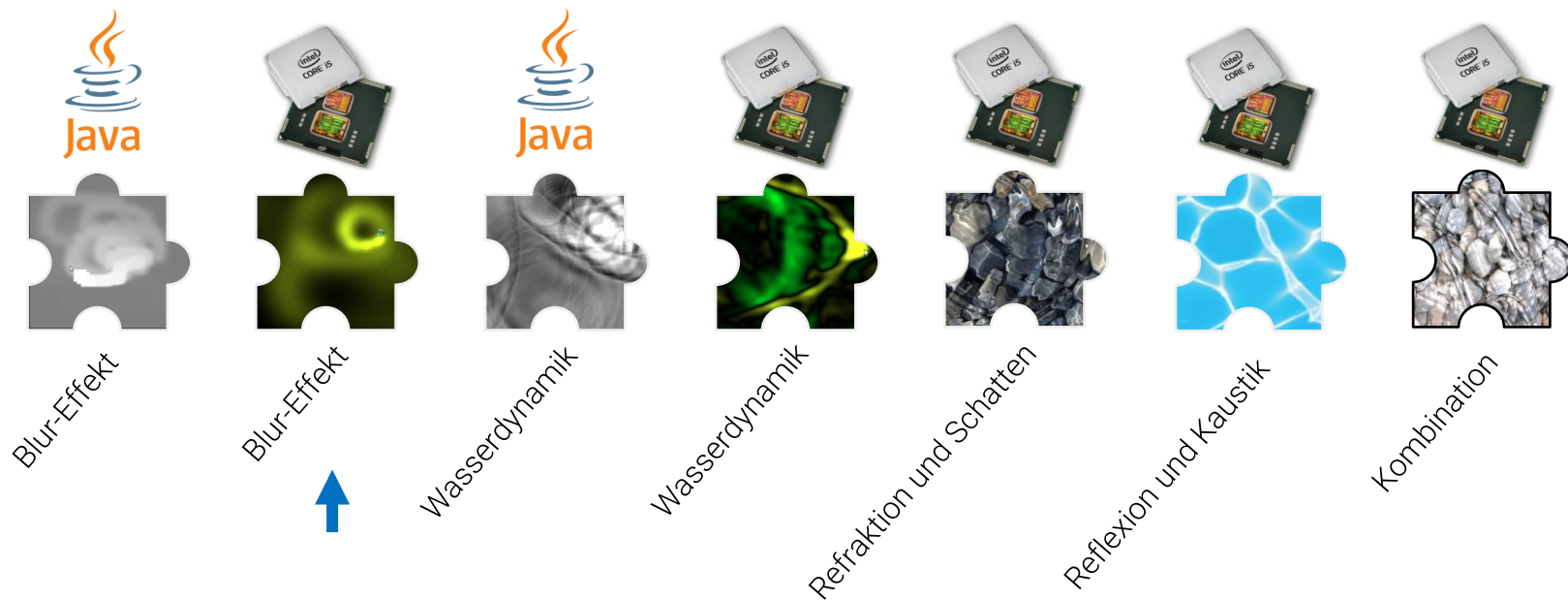
Ziele und Inhalt

- Interaktives Wasser – Projektetappen
- OpenGL-Renderpipeline (Khronos Group)
- OpenGL- und GLSL-Versionen
- GLSL-Besonderheiten und Datentypen
- Programmierbare Teile der Renderpipeline
- Projektstufe 2
- Shaderstrategie für Blur-Effekt
- Interaktion zwischen LWJGL und Shader
- Implementierung des Fragment-Shaders für Blur-Effekt
- Gauß-Verteilung statt Viereck
- Shader-Strategie für synthetischen Finger und Visualisierung
- Implementierung des Fragment-Shaders für Visualisierung
- Vom Java-Code zum Fragment-Shader
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.

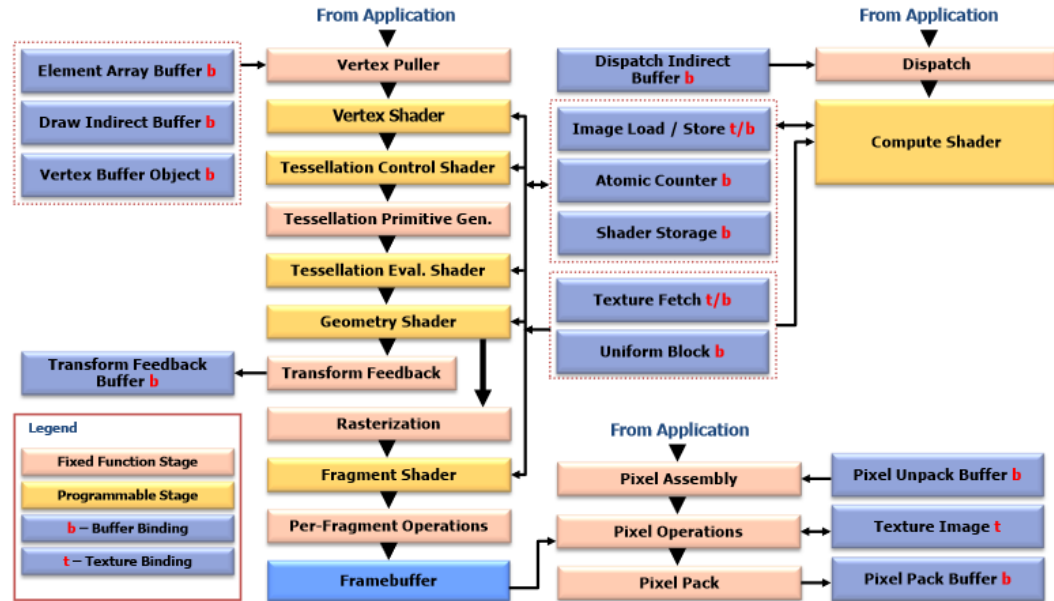
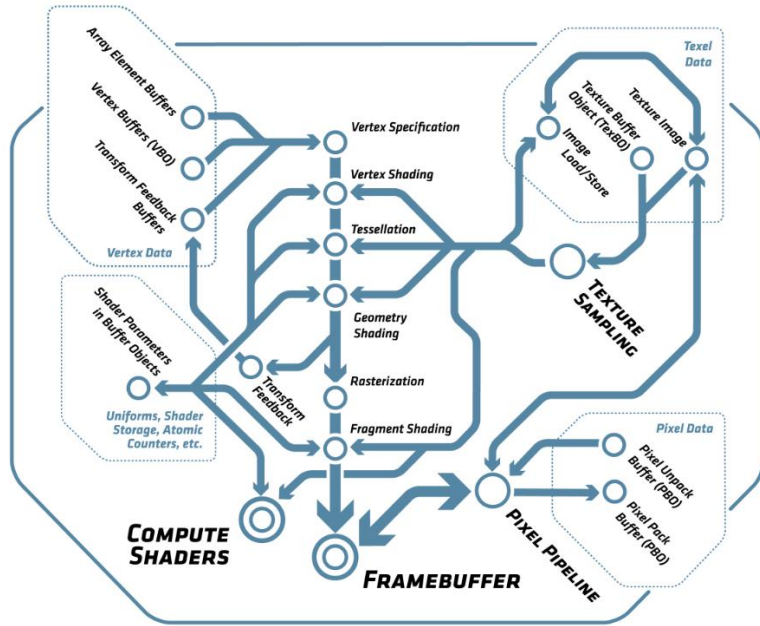




Schauen wir uns die [OpenGL-Renderpipeline](#) noch einmal genauer an ...

OpenGL-Renderpipeline (Khronos Group)

Die **vollständige OpenGL-Pipeline** der Version 4.6 von der Khronos Group ist für den Einstieg in die Shaderprogrammierung nicht besonders hilfreich [1].



[1] <https://www.khronos.org/registry/OpenGL/specs/gl/glspec46.compatibility.pdf> (1026 Seiten)

OpenGL- und GLSL-Versionen

Jede OpenGL-Version (seit Version 2.0) wurde mit einer korrespondierenden GLSL-Version veröffentlicht. [Besondere Erweiterungen](#) sind:

Geometry-Shader: Aus Primitiven können neue Primitive erstellt werden.

Tessellation-Shader: Flächen können in kleinere Flächen unterteilt werden.

Compute-Shader: Unabhängig von der Grafikpipeline können Daten im OpenGL-Kontext bearbeitet werden (General-Purpose Computation on Graphics Processing Units (kurz GPGPU)).

Um anzugeben, welche Version von GLSL zum Kompilieren und Linken eines Shaders verwendet werden soll, verwenden Sie diese [Direktive](#):

```
#version 140
... // Shadercode
```

Die Direktive **#version** muss als erstes im Shadercode stehen (mit Ausnahme von Leerzeichen und Kommentaren). Ohne Angabe wird die Version **1.10** angenommen.

OpenGL	GLSL	Besondere Erweiterung
2.0	1.10	
2.1	1.20	
3.0	1.30	
3.1	1.40	
3.2	1.50	Geometry-Shader
3.3	3.30	
4.0	4.00	Tessellation-Shader
4.1	4.10	
4.2	4.20	
4.3	4.30	Compute-Shader
4.4	4.40	
4.5	4.50	
4.6	4.60	

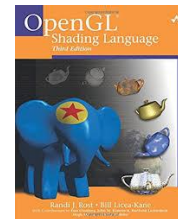
GLSL-Besonderheiten und Datentypen

Verkettung und Besonderheiten von **Konstrukturen** bei der Erzeugung:

```
vec2 zwei = vec2(-1.0, 1.5);  
vec4 vier = vec4(zwei, zwei);  
vec4 vierEinsen = vec4(1.0);
```

GLSL stellt eine Reihe von **Datentypen** zur Verfügung (siehe z. B. [1]):

```
// boolesche Datentypen  
bool, bvec2, bvec3, bvec4  
  
// ganzzahlige Datentypen  
int, ivec2, ivec3, ivec4, uint, uvec2, uvec3, uvec4  
  
// Gleitkommazahlen  
float, vec2, vec3, vec4, mat2, mat3, mat4  
  
// Datentypen für Texturen  
sampler2D
```

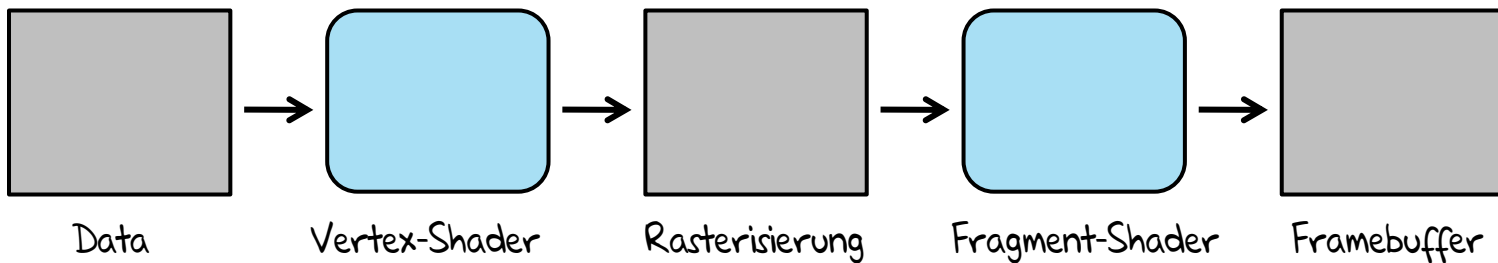
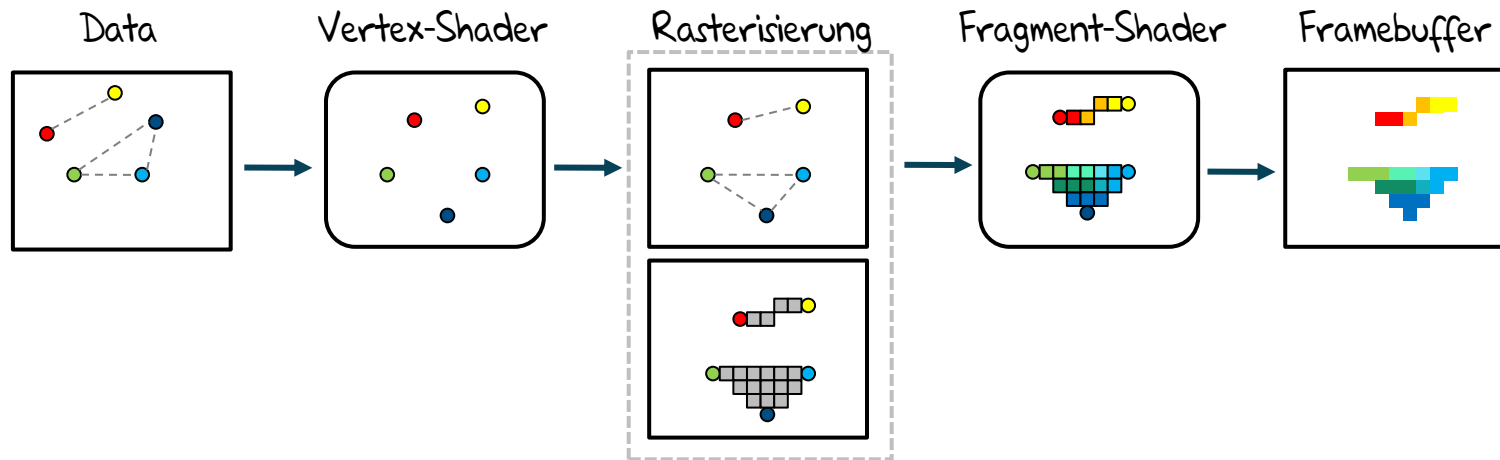


Rost R. J., et al.: „OpenGL Shading Language“, 3. Auflage, Addison-Wesley-Verlag, 2009 „orange book“

Aber es gibt in GLSL keine **Zeichenketten** (String), Pointer, byte, short oder long.

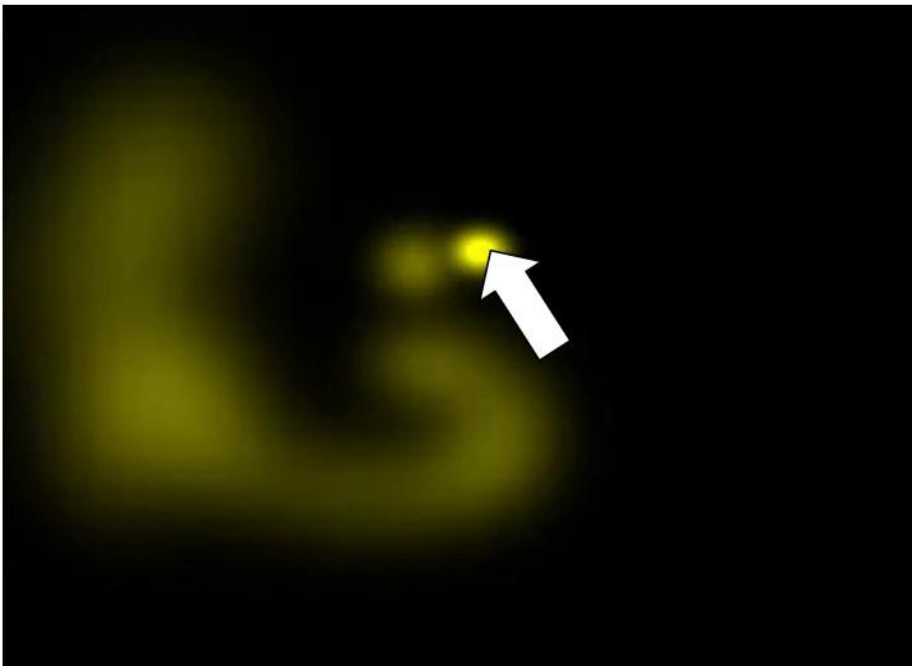
[1] [https://www.khronos.org/opengl/wiki/Data_Type_\(GLSL\)](https://www.khronos.org/opengl/wiki/Data_Type_(GLSL))

Programmierbare Teile der Pipeline



Projektziel – Stufe 2

In der zweiten Projektstufe wollen wir den **Blur-Effekt im Shader** realisieren. Dabei soll statt des weißen Vierecks ein **synthetischer Finger** zum Einsatz kommen. Der synthetische Finger ist eine weiche, gaußförmige Eingabequelle, die das harte weiße Viereck aus Stufe 1 ersetzt.



Projektstufe 2

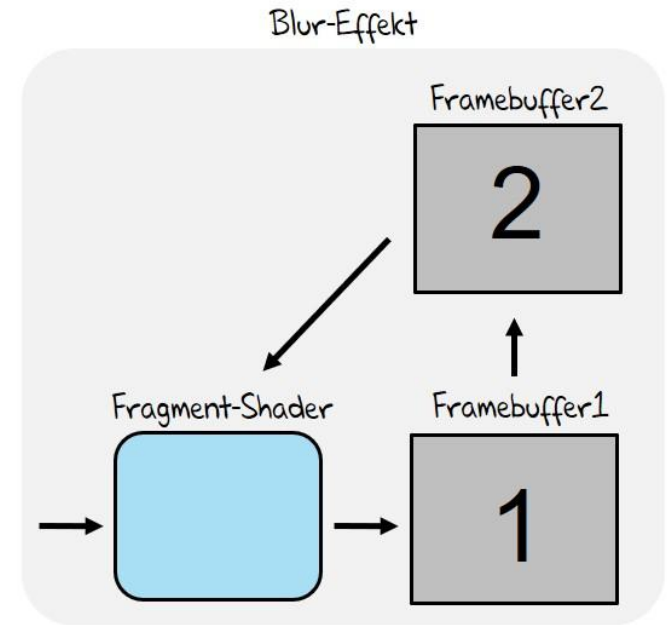
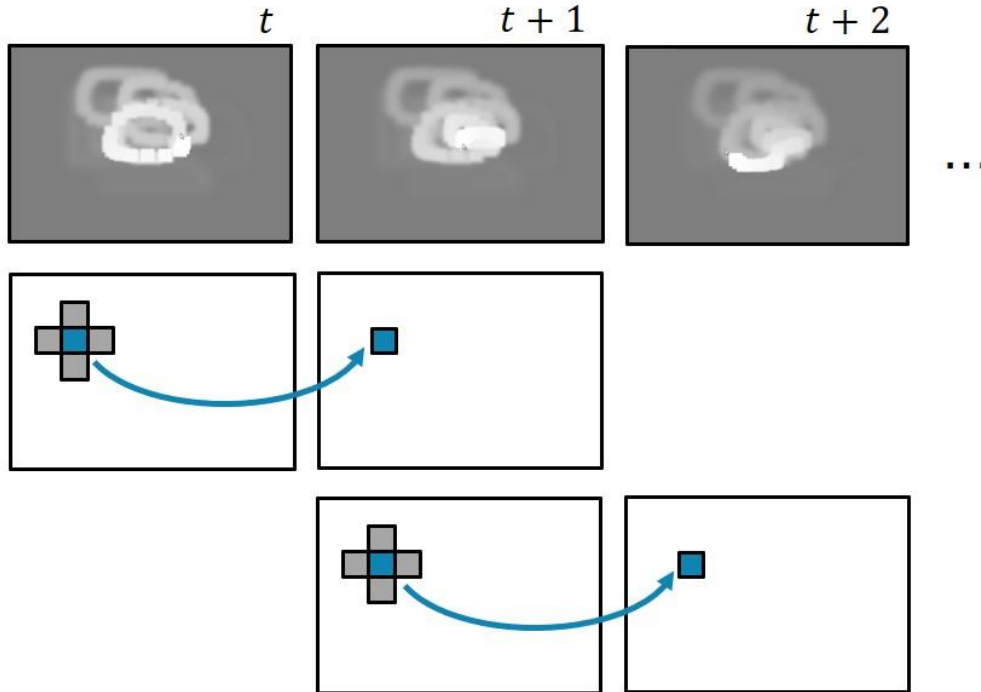
- wir haben einen schwarzen Hintergrundbereich
- mit einem gelben Vordergrundobjekt (synthetischer Finger)
- Implementierung als Shadercode



Geometrie bleibt hier unverändert ...
wir konzentrieren uns auf den **Fragment-Shader**

Shader-Strategie für Blur-Effekt

Um einen **kontinuierlichen Blur-Effekt** zu realisieren, benötigen wir also zwei Framebuffer. Der Vertex-Shader führt hier keine eigene Logik aus, die Geometrie bleibt unverändert. Der **Fragment-Shader** liest Daten aus **Framebuffer2** und speichert die berechneten Bilddaten in **Framebuffer1**.





Pixel-Shader versus **Fragment-Shader**



Interaktion zwischen LWJGL und Shader

Da bei dem Fragment-Shader alle Fragmente parallel verarbeitet werden, wir aber bei der Konvolution die Daten unserer **Nachbarn** erfragen wollen, müssen wir zunächst die **Schrittweite 1** für die Pixelabstände in Erfahrung bringen.

In der Variablen **s** wollen wir diesen Abstand, der ja für alle Pixel gleich ist, an den Shader übergeben. Der Abstand kann in x- und y-Richtung variieren, daher übergeben wir einen zweidimensionalen Vektor **s**. Zusätzlich übergeben wir in dem Parameter **tex2** noch die entsprechende Textur (den **Framebuffer2**), die durch den Datentyp **sampler2D** repräsentiert wird:

```
uniform sampler2D tex2;  
uniform vec2 s;  
  
void main (void) {  
    ... // sehen wir gleich noch  
}
```

Interaktion zwischen LWJGL und Shader

Damit die beiden Parameter als **uniform** übergeben werden können, müssen wir diese im Java-Quellcode erst einmal mit der Funktion **glGetUniformLocation** identifizieren, was der folgende Codeabschnitt zeigt:

```
int uniform_s      = glGetUniformLocation(programBlureffect, "s");  
int uniform_tex2   = glGetUniformLocation(programBlureffect, "tex2");  
  
glUseProgram(program_BlurEffect);           // Shaderprogramm aktivieren  
  
glUniform2f(uniform_s, 1.0f/WB, 1.0f/HB);   // ein vec2 wird erwartet  
glUniform1i(uniform_tex2, 0);               // der Ort einer gebundenen Texture  
                                           // wird erwartet  
  
glUseProgram(0);
```

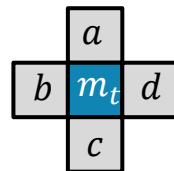


Implementierung des Fragment-Shaders für Blur-Effekt

Schauen wir uns jetzt den kompletten Fragment-Shadercode für den [gedämpften Mittelwertfilter](#) an:

```
uniform sampler2D tex2;
uniform vec2 s;

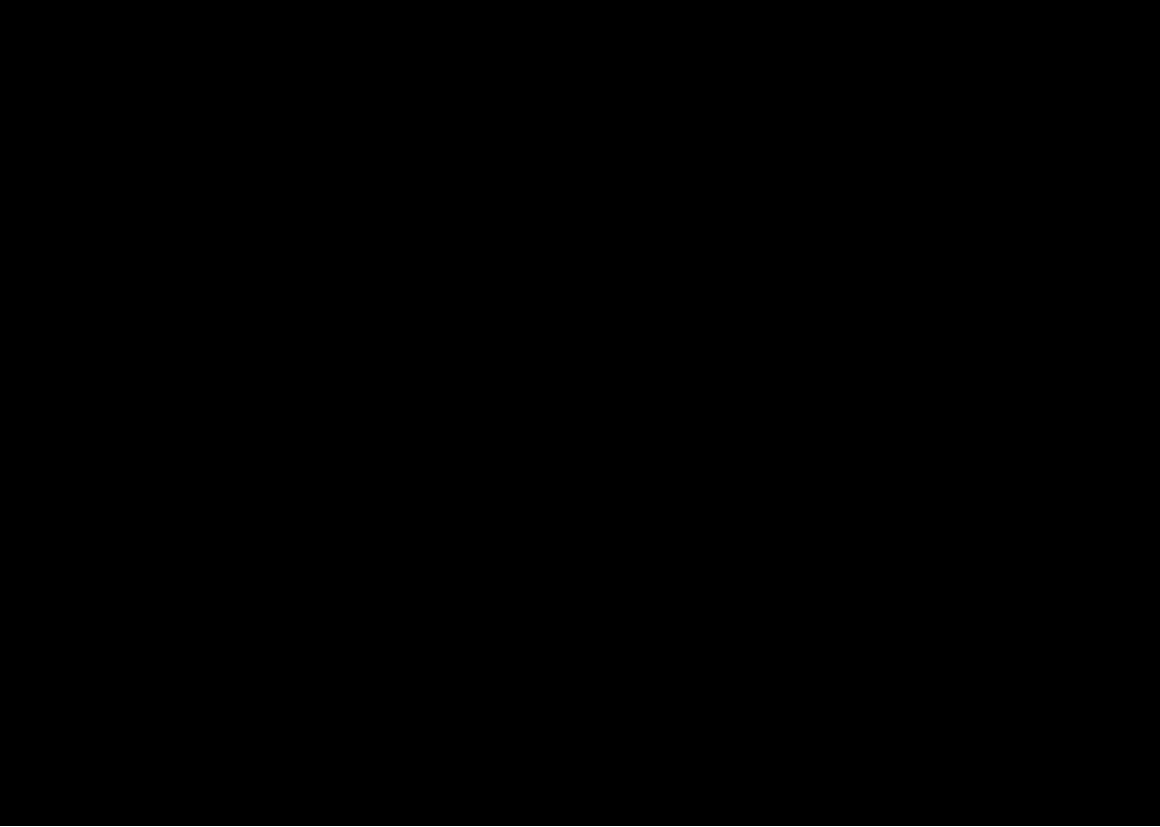
void main (void) {
    gl_FragColor = (
        texture2D(tex2, gl_TexCoord[0].st + vec2( s.x,    0)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2(-s.x,    0)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2(  0,   s.y)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2(  0,  -s.y)) +
        texture2D(tex2, gl_TexCoord[0].st) * 0.2 * 0.992;
    }
```



$$m_{t+1} = (a + b + c + d + m_t) \cdot 0.2 \cdot D$$

Der Faktor **0.992** wirkt als Dämpfung. Ohne diese Dämpfung würde die im Framebuffer gespeicherte Intensität nicht abklingen.

Implementierung direkt in Java



Programmcode:
kapitel02/BlurEffektCPU.java

Gaußverteilung statt Viereck

Da wir später das Gleiten der Finger im Wasser simulieren wollen und mit dem Viereck eine nicht sehr natürlich wirkende Eingabequelle haben, wollen wir kurz die [Gaußverteilung](#) als hervorragende Alternative besprechen.

```
final int WG = 32;
final int HG = 32;
FloatBuffer gaussBuffer = BufferUtils.createFloatBuffer(WG * HG);
final float SIGMA_SQ = WG / 1, S = 0.4f;

for (int y=0, Y=-HG/2; y<HG; y++, Y++)
    for (int x=0, X=-WG/2; x<WG; x++, X++)
        gaussBuffer.put(x+y*WG, S * (float)Math.exp(-0.5*(X*X+Y*Y)/SIGMA_SQ));
```

Synthetischer Finger

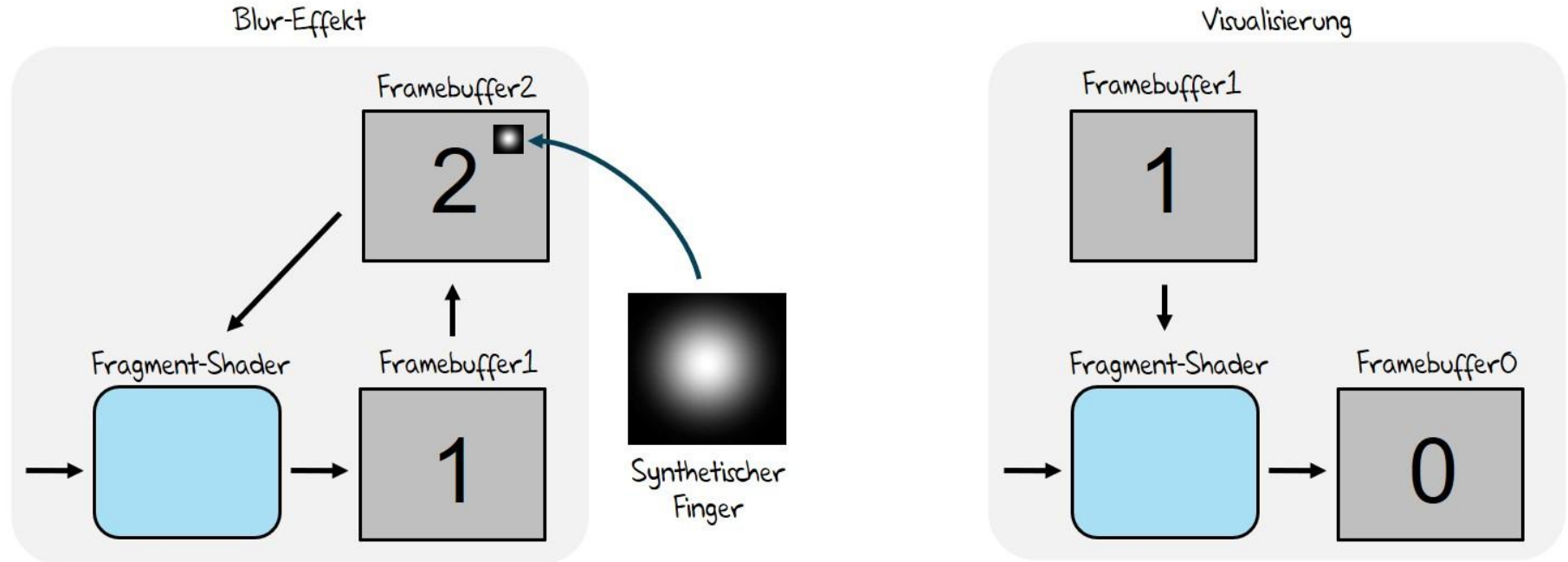


$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

Die Gaußverteilung erzeugt hohe Intensität im Zentrum und weiche Übergänge nach außen. Dadurch wirkt die Eingabe wie ein [Fingerabdruck im Wasser](#). Für die Implementierung wurden in Bezug auf Quadrat- und Wurzelfunktionen Vereinfachungen vorgenommen.

Shader-Strategie für synthetischen Finger und Visualisierung

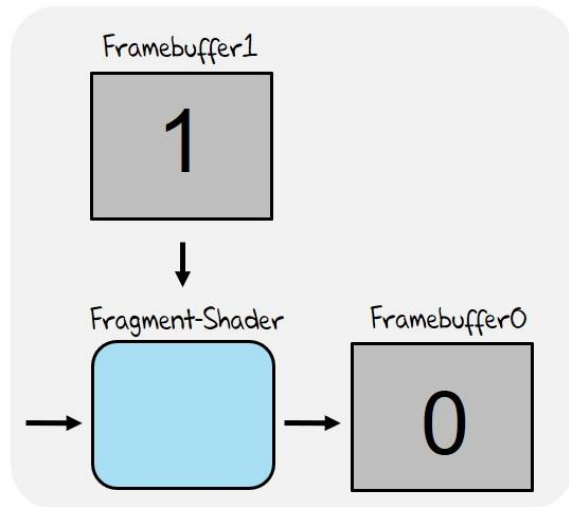
Um die [Visualisierung](#) etwas anschaulicher darzustellen, werden wir statt des Vierecks einen synthetischen Finger verwenden. Da wir bisher in einen unsichtbaren Framebuffer hineinschreiben, wollen wir diesen mit einem zweiten Shaderdurchlauf visualisieren.



Implementierung des Fragment-Shaders für Visualisierung

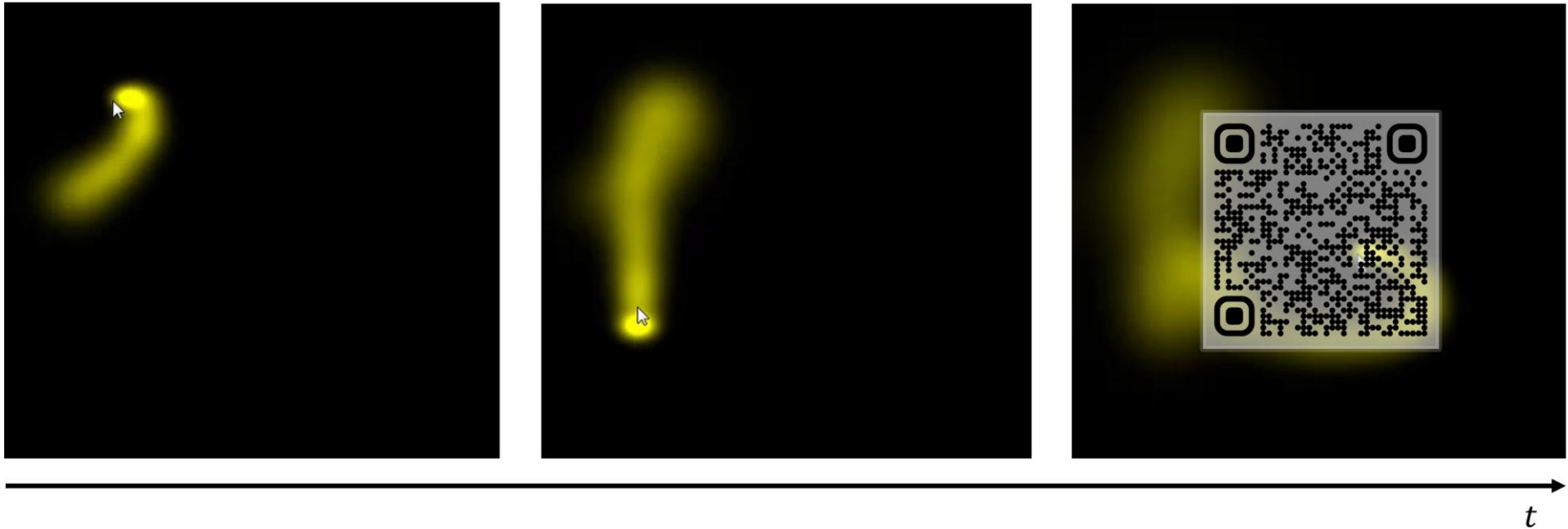
Der zweite Fragment-Shader zur Visualisierung ist ebenfalls sehr kurz. In diesem Fall wird durch die Textur **tex1** der Inhalt des **Framebuffer1** übergeben.

```
uniform sampler2D tex1;  
  
void main (void) {  
    gl_FragColor = texture2D(tex1, gl_TexCoord[0].st) *  
                    vec4(1.0, 1.0, 0.0, 1.0) * 5.0;  
}
```

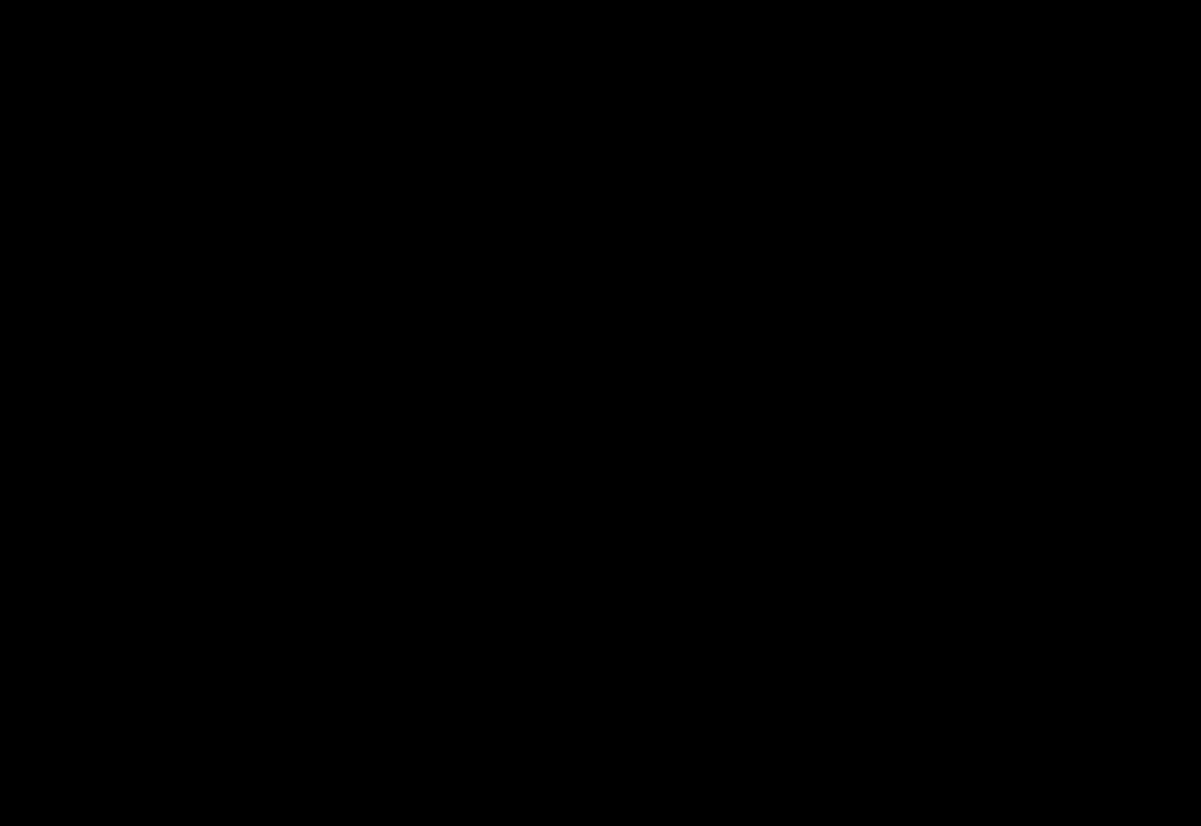


Ergebnis des Blur-Shaders

Wenn wir mit der linken Maustaste ein Muster in das Fenster malen, ist der gewünschte Effekt zu sehen. Der erste Shader berechnet den gedämpften Blur-Zustand in einem Framebuffer; der zweite Shader färbt diesen Zustand gelb ein und gibt ihn auf dem Bildschirm aus.



Implementierung im Shader



Programmcode:
kapitel02/BlurEffektGPU.java

Vom Java-Code zum Fragment-Shader

Schauen wir uns hier im Vergleich noch einmal zusammenfassend beide Varianten der Umsetzung an.

Bildarray t

```
for all target pixel (x, y) {
    target[x, y] = source[x-1, y] +
                  source[x+1, y] +
                  source[x, y] +
                  source[x, y-1] +
                  source[x, y+1];
    target[x, y] = G + target[x, y] * 0.2f * D;
}
```



Bildarray t+1

CPU-Variante

- Bilddaten liegen als Array im Hauptspeicher vor
- Pixel werden in einer verschachtelten Schleife verarbeitet
- Nachbarwerte werden über Indexzugriffe gelesen
- Ergebnis wird in ein neues Array geschrieben

Textur t

```
uniform sampler2D tex2;
uniform vec2 s;

void main (void) {
    gl_FragColor = (
        texture2D(tex2, gl_TexCoord[0].st + vec2( s.x,    0)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2(-s.x,    0)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2( 0,    s.y)) +
        texture2D(tex2, gl_TexCoord[0].st + vec2( 0,   -s.y)) +
        texture2D(tex2, gl_TexCoord[0].st)) * 0.2 * 0.992;
}
```



Textur t+1

GPU-Variante

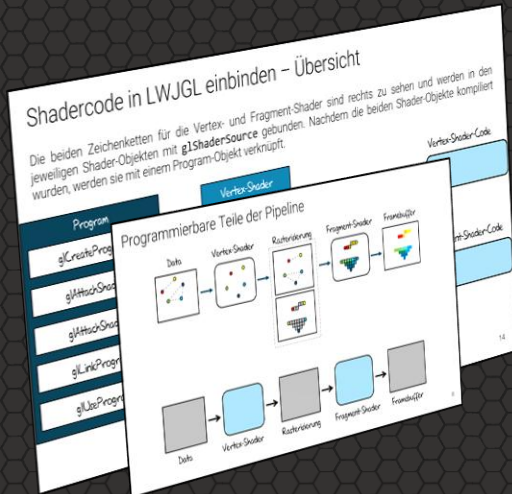
- Bilddaten liegen als Textur auf der GPU vor
- Fragment-Shader wird für jedes Fragment parallel ausgeführt
- Nachbarwerte werden über Textursampling gelesen
- Ergebnis wird direkt in den Framebuffer geschrieben



Damit haben wir unseren ersten
interaktiven Shader-Effekt realisiert!

Realisierung des Blur-Effekts mit einem Shader

Key Takeaways



- Blur entsteht durch Sampling benachbarter Texel im Fragment-Shader
- Framebuffer werden als Texturen in den Shader zurückgeführt (alten Zustand lesen/neuen Zustand) schreiben
- Konvolution (Mittelwertfilter) bildet die Grundlage des Effekts
- Dämpfungsfaktor verhindert Aufsummieren der Intensität
- Effekt läuft vollständig im Fragment-Shader (parallel auf GPU)
- Zweiter Shaderdurchlauf zur Visualisierung des Ergebnisses
- Gaußverteilung liefert weichere, realistischere Eingaben als ein Rechteck

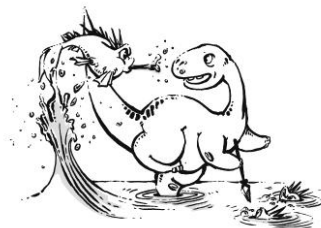
Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Welche Aufgaben können mit dem Geometry-Shader, dem Tessellation-Shader und dem Compute-Shader gelöst werden?
- (2) Welche der folgenden Programmzeilen in GLSL entsprechen nicht der gültigen Syntax und warum?

```
vec2 pos  = vec2(1.0, 2.0);  
vec3 pos2 = vec3(pos.x, pos.g, -1.0);  
vec3 pos3 = pos.xyb;  
vec3 pos4 = vec3(1.0);  
vec3 pos4 = vec3(1.0, 2.0);
```

- (3) Vergleichen Sie die vereinfachte OpenGL-Pipeline mit der OpenGL-Renderpipeline (Khronos Group).
- (4) Führen Sie das Programm **kapitel02/BlurEffektGPU.java** aus und beschäftigen Sie sich mit dem dort vorgestellten Programmcode zur Realisierung des Blur-Effekts.



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Verändern Sie den Dämpfungsfaktor im Blur-Shader und beobachten Sie, wie lange die Spur sichtbar bleibt.

Aufgabe 2) [Java/GLSL] Erweitern Sie den Mittelwertfilter von 5 auf 9 Samples.

Aufgabe 3) [Java/GLSL] Ändern Sie die Visualisierungsfarbe von Gelb auf Blau oder Cyan.

Aufgabe 4) [Java] Ersetzen Sie den synthetischen Finger durch eine größere oder kleinere Gaußverteilung.



Vielen Dank für die Aufmerksamkeit